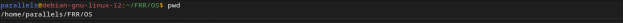
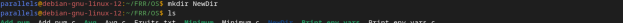
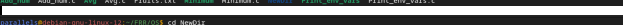
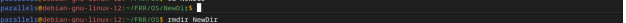
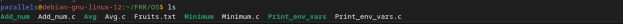
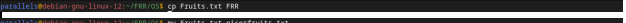
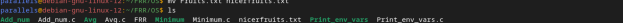
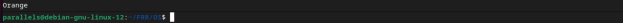
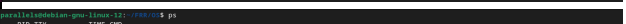
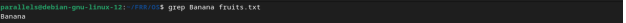
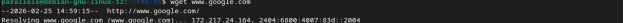
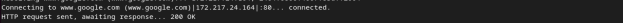
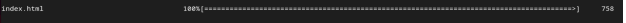


OS LAB 1 ACTIVITY			
1.1 BASIC LINUX TERMINAL COMMANDS			
SUBMITTED BY: FRANCIS REUBEN R 18UA2SC00168 B - 50			
SUBMITTED TO: PROF. PRIYANKA S			
Sl.no	Commands	Description	SCREENSHOTS
1	ls	Displays information about files in the current directory.	
2	pwd	Displays the current working directory.	
3	mkdir	Creates a directory.	
4	cd	To navigate between different folders.	
5	rmdir	Removes empty directories from the directory lists.	
6	cp	Copy files from one directory to another.	
7	mv	Rename and Replace the files	
8	rm	Delete files	
9	uname	Command to get basic information about the OS	
10	locate	Find a file in the database.	
11	touch	Create empty files	
12	ln	Create shortcuts to other files	
13	cat	Display file contents on the terminal	
14	clear	Clear terminal	
15	ps	Display the processes in the terminal	
16	man	Access the manual for all Linux commands	
17	grep	Search for a specific string in an output	
18	echo	Print string or text to the terminal	
19	wget	download files from the internet.	
20	whoami	Displays the current user's name	
21	sort	sort the file content	
22	cal	View the Calendar in the terminal	
23	whereis	View the exact location of any command typed after this command	
24	df	Check the details of the file system	
25	wc	Check the lines, word count, and characters in a file using different options	

1.2) Execute the C program using Command Line Arguments.

i. Add numbers

```
#include <stdio.h>
#include <stdlib.h>
int Add_num(int nParams, char *argv[]);
int main(int argc, char *argv[])
{
    printf("Calculate the sum of numbers entered by the user!!!\n");
    if (argc < 2)
    {
        printf("Error: Insufficient parameters!!!\n");
        printf("Give at least one or more valid integer numbers as parameters.\n");
        return -1;
    }
    int nParams = argc - 1;
    int sum = 0;
    for (int i = 0; i < argc; i++)
    {
        printf("argv[%d] = %s\n", i, argv[i]);
    }
    sum = Add_num(nParams, argv);
    return 0;
}

int Add_num(int nParams, char *argv[])
{
    char *pParam;
    int sum = 0;
    int num;

    pParam = argv[1];
    for (int i = 1; i <= nParams; i++)
    {
        pParam = argv[i]; // Copy the pointer to the parameter passed
        num = atoi(pParam);
        sum += num;
        printf("pParam = %s, sum = %d\n", pParam, sum);
    }
    printf("The sum of numbers given is: %d\n", sum);

    return 0;
}
```

```
parallel@debian-gnu-linux-12:~/FRR/0$ ls
Add_num.c  Avg  Fr  FRR  index.html  Minimum.c  Print_env_vars.c
Add_num.c  Avg.c  Fr  fruits.txt  Minimum  Print_env_vars
parallel@debian-gnu-linux-12:~/FRR/0$ ./Add_num 10 -10 20
Calculate the sum of numbers entered by the user!!!
argv[0] = ./Add_num
argv[1] = 10
argv[2] = -10
argv[3] = 20
pParam = 10, sum = 10
pParam = -10, sum = 0
pParam = 20, sum = 20
The sum of numbers given is: 20
```

ii. Find the minimum from the given numbers

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h> // required in linux
int f_Min(int nParams, char *argv[]);
int main(int argc, char *argv[])
{
    printf("Find the minimum of the numbers entered by the user!!!\n");
    if (argc < 2)
    {
        printf("Error: Insufficient parameters!!!\n");
    }
}
```

```
parallel@debian-gnu-linux-12:~/FRR/0$ ./Minimum 10 -999 40
Find the minimum of the numbers entered by the user!!!
argv[0] = ./Minimum
argv[1] = 10
argv[2] = -999
argv[3] = 40
The min now is 10
The min now is -999
The min now is -999
The minimum of the numbers given is: -999
```

